

Digital Image Processing

Beginner Friendly Notes • Units A to E • Exam Ready

Explained simply — like you're 15, with real-life examples

Unit	Title	What it's about (in one line)
A	Digital Image Fundamentals	How a photo is stored as numbers in a computer
B	Fourier Transform & DFT	Breaking an image into waves — like music frequencies
C	Image Enhancement	Making dark/blurry photos look better
D	Restoration & Compression	Fixing damaged photos & making files smaller (JPEG/MP4)
E	Segmentation & Representation	Teaching computers to find objects inside a photo

UNIT A — Digital Image Fundamentals & Transform

Topic A1: How We See + How Images Are Stored

■ First — What IS a Digital Image?

A digital image is just a **big grid of tiny squares called pixels**. Each pixel has a number that tells the computer how bright it is. That's it. Your 12MP phone camera = 12 million of these tiny squares.

■ *Real-life example: Think of a mosaic art — it's made of small tiles. Each tile = one pixel. From far away it looks like a photo. Up close you see the tiles.*

■ How the Human Eye Works (Visual Perception)

Part	What it does	Real-life comparison
Lens	Focuses light onto the retina	Like a camera lens
Retina	Converts light into signals	Like the camera sensor/film
Cones (~7 million)	See colour, work in bright light	Like colour TV pixels
Rods (~125 million)	See in low light, detect motion	Like night-vision
Fovea	Centre of retina — sharpest vision	Like the centre of your phone screen

3 Cool tricks your eye plays on you (important for exam!):

- **Brightness adaptation:** Your eye adjusts to dim/bright rooms. Same way phone screen auto-brightness works.
- **Simultaneous contrast:** A grey box looks darker on white background, lighter on black. Your brain changes what it 'sees' based on surroundings.
- **Mach band effect:** At edges where dark meets light, your eye sees a fake extra-bright stripe and a fake extra-dark stripe. Your brain exaggerates edges.

■ Sampling — Converting Real World to Numbers

Sampling = deciding *how many pixels* to use (left-right and up-down). More pixels = higher resolution = sharper image.

Quantization = deciding *how many brightness levels* to use per pixel. More levels = smoother gradients, less banding.

■ *Analogy: Sampling = how many tiles you use in your mosaic. Quantization = how many shades of colour paint you have.*

Key Formulas — MEMORISE THESE:

Grey levels = 2^k (k = number of bits per pixel)

Storage needed = $M \times N \times k$ bits (M=rows, N=columns, k=bits)

Example: An 8-bit image has $2^8 = 256$ grey levels (0=black, 255=white). A 1024x1024 pixel, 8-bit image needs $1024 \times 1024 \times 8 = 8,388,608$ bits = about 1MB.

■ **Aliasing:** If you use too FEW pixels (under-sample), the image looks jagged/staircase. This is why Instagram photos look bad when you screenshot and re-screenshot them many times.

Nyquist Rule: Sample at least 2x faster than the detail you want to capture.

Topic A2: Relationships Between Pixels

■ Which pixels are 'neighbours'?

Imagine you're standing on a chess board square. Your neighbours are the squares around you.

Type	Which squares count?	How many?	Used for?
4-neighbours (N4)	Up, Down, Left, Right only	4	Simple connectivity
Diagonal (ND)	4 diagonal corners only	4	Corner detection
8-neighbours (N8)	All 8 surrounding squares	8	Most image processing

Connectivity = two pixels are 'connected' if they are neighbours AND have similar brightness values.

Region = a group of connected pixels (like selecting an object in Photoshop).

Boundary = the pixels at the edge of a region (where your region meets 'outside').

Distance Formulas — 3 ways to measure distance between pixel (x,y) and (s,t):

Distance Type	Formula	Analogy
Euclidean (straight line)	$\sqrt{(x-s)^2 + (y-t)^2}$	As the crow flies
City-block D4	$ x-s + y-t $	Taxi driving in NYC grid — only left/right/up/down
Chessboard D8	$\max(x-s , y-t)$	King moving in chess — diagonals count as 1 step

Topic A3: Geometric Transformations

■ Geometric Transformations — Moving/Reshaping Images

These are operations that change **where pixels are** in the image — not their brightness. Think Instagram filters that rotate/zoom/warp your photo.

Transform	What it does	Formula	Real-life
Translation	Slide image up/down/left/right	$x' = x+tx, y' = y+ty$	Moving a sticker on screen
Scaling	Zoom in or out	$x' = sx*x, y' = sy*y$	Pinch-to-zoom on phone
Rotation	Spin the image by angle θ	$x' = x \cos\theta - y \sin\theta$ $y' = x \sin\theta + y \cos\theta$	Rotating a photo in gallery
Shearing	Slant/tilt the image	$x' = x + shx*y$	Italic text effect
Affine	Combo of above (rotate+scale+translate)	$M * p = A*p + b$	Used in image stitching (panorama)

■■ **Interpolation:** After transforming, some pixels end up 'between' grid positions. We need to estimate the colour there. 3 methods: **Nearest-neighbour** (fast, blocky), **Bilinear** (smooth, common), **Bicubic** (best quality, slowest).

UNIT B — Introduction to Fourier Transform and DFT

Topic B1: Fourier Transform, DFT & FFT

■ What IS the Fourier Transform? — Start Here!

Imagine you press a chord on a piano — multiple notes play at once. The **Fourier Transform** is like an app that listens to that combined sound and tells you *exactly which individual notes are in it and how loud each one is*.

For images: instead of sound waves, we have **brightness patterns** (light/dark stripes at different spacings). The Fourier Transform breaks an image into these spatial frequency patterns.

■ *Low frequencies = big smooth colour regions (background, sky). High frequencies = sharp edges, fine details, noise.*

1D DFT Formula (for 1 row of pixels):

$$F(u) = \text{SUM from } x=0 \text{ to } N-1 \text{ of: } f(x) * e^{(-j*2*\pi*u*x/N)}$$

$$\text{Inverse DFT (to get back the original): } f(x) = (1/N) * \text{SUM } F(u) * e^{(j*2*\pi*u*x/N)}$$

2D DFT Formula (for a full image):

$$F(u,v) = \text{SUM}_x \text{ SUM}_y f(x,y) * e^{(-j*2*\pi*(ux/M + vy/N))}$$

Don't panic about the formula! Just know: **f(x,y)** = pixel values. **F(u,v)** = frequency content. u,v are frequency coordinates (like musical notes).

8 Important Properties of 2D Fourier Transform:

Property	Plain English Meaning	Formula hint
Linearity	FT of (A+B) = FT(A) + FT(B)	$FT\{af+bg\} = aF+bG$
Translation	Shifting image = phase change in freq domain	$f(x-x_0, y-y_0)$ shifts $F(u,v)$
Periodicity	DFT output repeats periodically	$F(u,v) = F(u+M, v+N)$
Conjugate symmetry	For real images, $F(-u, -v) = F^*(u, v)$	Saves computation
Convolution theorem	Blurring in image = multiplying in freq	$f * h \implies F \times H$ (KEY!)
Correlation	Cross-correlation = mult by conjugate	$f \star g \implies F^* \times G$
Rotation	Rotating image also rotates its freq spectrum	Rotate $f \implies$ rotate F
Distributivity	FT spreads over addition	$FT\{f+g\} = FT\{f\} + FT\{g\}$

■ FFT — Fast Fourier Transform

Regular DFT on N points takes **N×N calculations** — extremely slow for large images.

FFT (invented by Cooley & Tukey, 1965) reduces this to **N × log(N) calculations** by reusing calculations cleverly.

$$\text{DFT: } O(N^2) \rightarrow \text{FFT: } O(N \log N)$$

■ *Example: For N=1024 pixels: DFT = 1,048,576 ops. FFT = only 10,240 ops. That's 100x faster! This is why real-time video processing is possible.*

Separability: 2D FFT = apply 1D FFT to all rows, then 1D FFT to all columns. Two passes only!

Topic B2: Walsh-Hadamard Transform (WHT)

■ Walsh-Hadamard — The 'No Math' Transform

Instead of using sin/cos waves (like Fourier), WHT uses only **+1 and -1** values. No complex numbers, no trigonometry — just additions and subtractions!

■ *Analogy: Instead of a piano (smooth waves), WHT uses an on/off light switch. Simpler but still useful for compression.*

- Basis functions called **Walsh functions** — rectangular on/off patterns (not smooth waves).
- **Hadamard matrix** H built by doubling: $H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, then $H_4 = H_2 \times H_2$, and so on.
- Formula: $F = (1/N) * H * f * H$ (apply H to rows then columns)
- **Sequency** = number of sign changes in a Walsh function (like 'frequency' for FFT).
- **Fast WHT**: $O(N \log N)$ — same speed advantage as FFT.
- **Key use**: Image compression in low-power devices — no floating point needed, just +/-1 arithmetic.

Topic B3: Discrete Cosine Transform (DCT)

■ DCT — The JPEG Secret Weapon

DCT is the transform inside **every JPEG photo** on the internet. It's like Fourier Transform but uses only **cosine waves** (no complex numbers, all real).

■ *Why is DCT special? It packs most of the image's important information into just the first few numbers. The rest are near-zero and can be thrown away. That's how JPEG makes files 10x smaller!*

1D DCT Formula:

$$C(u) = \alpha(u) * \sum_{x=0}^{N-1} f(x) * \cos[(2x+1)*u*\pi / 2N]$$

$$\alpha(0) = \sqrt{1/N}, \alpha(u) = \sqrt{2/N} \text{ for } u > 0$$

Concept	Explanation
DC coefficient C(0)	The average brightness of the block — the most important value
AC coefficients C(1), C(2)...	Fine details and edges — higher index = finer detail
Energy compaction	Most energy is in C(0) and first few values — rest are ~0
Why DCT over DFT?	No ringing artefacts at block edges (Gibbs phenomenon avoided)
8x8 blocks in JPEG	Image is divided into 8x8 blocks, DCT applied to each block

UNIT C — Image Enhancement Techniques

Topic C1: Grey Level Transformations & Histogram Equalization

■ Grey Level Transformations — Changing Brightness

These are simple operations where you take each pixel's brightness value (r) and map it to a new value (s) using a function: $s = T(r)$

■ Think of it like Instagram filters — they run every pixel through a brightness formula.

Technique	Formula	What it does	Real-life use
Image Negative	$s = (L-1) - r$ ($L=256$ for 8-bit)	Flips black to white, white to black	Like photo negative in old film
Log Transform	$s = c * \log(1 + r)$	Brightens dark areas, compresses bright	Showing X-rays — reveals detail in shadows
Gamma (Power-Law)	$s = c * r^{\gamma}$	$\gamma < 1$: brightens, $\gamma > 1$: darkens	Monitor/TV colour calibration
Contrast Stretch	Expand range $[a,b]$ to $[0, 255]$	Makes faded photos vivid	Restoring old scanned photos
Thresholding	$s=255$ if $r \geq T$, else $s=0$	Makes everything black or white	Scanning text documents

■ What is a Histogram?

A **histogram** is a bar chart that shows how many pixels have each brightness level. X-axis = brightness (0 to 255). Y-axis = how many pixels have that brightness.

Histogram shape	What it means	Problem
Bars clustered at LEFT	Dark image	Too dark — hard to see detail
Bars clustered at RIGHT	Bright/washed out image	Overexposed
Tall narrow cluster in MIDDLE	Low contrast	Looks flat and grey
Bars spread evenly	Good contrast	Clear, detailed image

■ Histogram Equalization (HE) — Auto Contrast Fix

Goal: Automatically improve contrast by spreading pixel values evenly across 0-255. No manual adjustment needed!

■ Analogy: You have 256 students (brightness levels) squeezed into 3 classrooms. HE redistributes them so each classroom has roughly equal students.

HE Formula:

$$s_k = (L-1) * CDF(r_k)$$

where $CDF(r_k)$ = sum of all histogram values from 0 to k , divided by total pixels

Steps to do HE:

1. Count how many pixels have each grey level → this is the histogram $h(r_k)$.
2. Divide by total pixels N to get probability: $p(r_k) = h(r_k) / N$.
3. Calculate cumulative sum (CDF): add up probabilities from 0 to k .
4. Multiply CDF by $(L-1)=255$ and round → this is the new pixel value.

■■ Drawback: HE can over-amplify noise and look unnatural on some images. CLAHE (Contrast Limited Adaptive HE) fixes this by applying HE to small local tiles.

Topic C2: Image Subtraction, Averaging & Smoothing Filters

■ Arithmetic Operations on Images

Operation	Formula	Real-life use	Key benefit
Addition	$g = f_1 + f_2$	Adding text/watermark to image	Combine two images
Subtraction	$g = f_1 - f_2$	Find differences between two frames	Motion detection in CCTV
Averaging	$g = (f_1 + f_2 + \dots + f_K) / K$	Average K photos of same scene	Removes random noise
Multiplication	$g = f * \text{mask}$	Keep only selected region	Select/crop an object

■ *Image Averaging example: Your phone takes burst photos and averages them for Night Mode. Random noise in each photo is different, so averaging cancels it out. K photos → noise reduced by factor $\sqrt{K}$.*

■ Spatial Filtering — How Blur & Sharpen Work

A **filter/kernel** is a small grid of numbers (e.g. 3x3). You slide it over every pixel, multiply the kernel numbers with the pixel values underneath, sum them all up → that's the new pixel value. This operation is called **convolution**.

Output $g(x,y) = \text{SUM over kernel: } w(s,t) * f(x+s, y+t)$

■ *Think of a kernel as a 'stamp'. You stamp it on every pixel and the numbers in the stamp decide what effect you get.*

Filter	Kernel values	Effect	Good for
Box/Average	All values = $1/9$ (for 3x3)	Blurs the image	Fast simple blur
Gaussian	Bell-curve weights, centre heaviest	Natural-looking smooth blur	Noise removal, pre-processing
Median	No kernel — replace with median value	Removes isolated noise spots	Salt-and-pepper noise
Max filter	Replace with max value in neighbourhood	Brightens image	Remove dark noise (pepper)
Min filter	Replace with min value in neighbourhood	Darkens image	Remove bright noise (salt)

Topic C3: Sharpening Filters, Laplacian & Frequency Domain Filtering

— ■ Sharpening — Making Edges Crisp

Sharpening does the opposite of blurring — it **enhances edges and fine detail**. It works by detecting where brightness changes fast (edges) and amplifying those changes.

■ *Analogy: Blurring = spreading butter on toast (smooth). Sharpening = drawing the outline of the toast with a marker (highlights edges).*

Gradient Operators — Detect Edges using 1st Derivative:

Operator	Kernels	What it finds	Formula
Sobel	$G_x = [-1, 0, 1; -2, 0, 2; -1, 0, 1]$ $G_y = [-1, -2, -1; 0, 0, 0; 1, 2, 1]$	Horizontal & vertical edges	Edge strength = $\sqrt{G_x^2 + G_y^2}$
Prewitt	Same as Sobel but all weights equal	Horizontal & vertical edges	Same as Sobel
Roberts	2x2 diagonal kernels	Diagonal edges	Fast but noisy

Laplacian — Detect Edges using 2nd Derivative:

The Laplacian measures how quickly the gradient is changing — it fires at edges but also at noise.

Laplacian kernel (4-neighbour): [0, 1, 0; 1, -4, 1; 0, 1, 0]

Laplacian kernel (8-neighbour): [1, 1, 1; 1, -8, 1; 1, 1, 1]

Laplacian sharpening: $g(x,y) = f(x,y) - c * \text{Laplacian}(f)$ [$c=1$ usually]

Unsharp Masking: $g = f + k * (f - \text{blurred}_f)$ [k controls sharpness amount]

■ Unsharp masking is literally in Photoshop and Lightroom — now you know what it does!

■ Frequency Domain Filtering — Filter with Fourier

The Big Idea (Convolution Theorem):

Instead of sliding a kernel over every pixel (slow), you can: 1) Convert to frequency domain with FFT, 2) Multiply by a filter, 3) Convert back. Same result, often faster for large kernels!

Steps: $\text{FFT}(\text{image}) \rightarrow \text{multiply by } H(u,v) \rightarrow \text{Inverse FFT} \rightarrow \text{filtered image}$

Filter Name	$H(u,v)$ formula	Effect	Artefact?
Ideal Low-Pass (LPF)	$H=1$ if $D \leq D_0$, else 0	Blurs image, removes high-freq noise	Ringing/halo artefacts
Butterworth LPF	$1 / (1 + (D/D_0)^{2n})$	Smooth blur, gradual cutoff	Slight ringing
Gaussian LPF	$e^{-(D^2 / 2 \cdot \sigma^2)}$	Smoothest blur, no artefacts	None — best choice!
Ideal High-Pass (HPF)	$H=0$ if $D \leq D_0$, else 1	Sharpens, shows only edges	Ringing artefacts
Butterworth HPF	$1 / (1 + (D_0/D)^{2n})$	Smooth sharpening	Minimal
Gaussian HPF	$1 - \text{Gaussian LPF}$	Natural sharpening	None

D = distance of (u,v) from centre of freq spectrum. D_0 = cut-off frequency you choose. n = order (higher n = sharper cutoff).

UNIT D — Image Restoration and Image Compression

Topic D1: Image Degradation Model, Noise & Restoration Filters

■ Why Does an Image Get Degraded?

When you take a photo, the result might be blurry, noisy, or distorted. This is called **degradation**. The goal of Image Restoration is to undo this damage and recover the original image.

■ *Analogy: You send a message through a bad phone line and it arrives garbled. Restoration = trying to figure out the original message.*

The Degradation Model:

Observed image: $g(x,y) = h(x,y) \text{ CONVOLVED WITH } f(x,y) + \text{noise}(x,y)$

In frequency domain: $G(u,v) = H(u,v) * F(u,v) + N(u,v)$

g = what you see (blurry/noisy). f = original (what you want). h = degradation function (blur kernel). n = noise.

Types of Noise:

Noise Type	How it looks	Caused by	Best fix
Gaussian	Random smooth variation everywhere	Electronic sensor heat	Gaussian/average filter
Salt & Pepper	Random black & white dots scattered pixels	Dead pixels, transmission errors	Median filter
Uniform	All grey levels equally likely	Quantization	Averaging filter
Rayleigh	Bright spots, right-skewed	Radar/range imaging	Specific filters
Exponential	Decays from peak	Laser speckle	Specialised
Impulse	Sudden bright/dark spikes	Bit errors during transmission	Median filter

Restoration Filters (how to UNdo the blur/noise):

Filter	Formula	Simple explanation	Problem
Inverse Filter	$F_{\text{hat}} = G / H$	Direct undo of blurring	Amplifies noise enormously — usually unusable
Wiener Filter	$F_{\text{hat}} = [H^* / (H ^2 + S_n/S_f)] * G$	Balances deblurring vs noise	Need to know noise/signal power ratio
Constrained LS	$F_{\text{hat}} = [H^* / (H ^2 + \gamma P ^2)] * G$	Minimize roughness while matching data	Iterate gamma parameter iteratively

■ *Wiener filter is like a 'smart' undo — it knows some noise is present and doesn't try to perfectly reverse every tiny fluctuation (which would amplify noise).*

Topic D2: Image Compression — Lossless & Lossy

■ Why Compress? The Problem of File Size

A raw 12MP photo (4000x3000 pixels, 3 colour channels, 8 bits each) = $4000 \times 3000 \times 3 \times 8 =$ **288 million bits = 36 MB per photo**. Your phone would fill up after 28 photos! Compression fixes this.

■ *3 types of redundancy we exploit: (1) Coding redundancy — some values appear more often than others. (2) Spatial redundancy — neighbouring pixels are often similar. (3) Psychovisual redundancy — our eyes can't notice certain details anyway.*

Compression Ratio = Original size / Compressed size

LOSSLESS Compression (no data lost — can fully reconstruct original):

Method	How it works	Key idea	Used in
Huffman Coding	Frequent values get shorter binary codes like Morse code — E gets short code	Like Morse code — E gets short code	ZIP, PNG, JPEG header
Run-Length (RLC)	Replace long runs of same pixel with (value, count) — AAAAAAA, store (A, 7)	Instead of AAAAAAA, store (A, 7)	BMP, fax machines, PCX
LZW	Build a dictionary of patterns while encoding	No need to send dictionary separately	GIF, TIFF, PDF
Bit-Plane Coding	Split into 8 layers (1 per bit), compress each	MSB layer has most info	Medical imaging
DPCM	Store only the DIFFERENCE between pixels	Difference is much smaller than numbers	JPEG DC coefficients, audio
Arithmetic Coding	Encode whole message as one decimal number	Approaches theoretical limit	JPEG2000, modern codecs

LOSSY Compression (throws away some data — smaller file, cannot fully reconstruct):

Method	How it works	Key idea	Used in
Transform Coding	DCT -> Quantize -> Entropy code	Throw away high-freq coefficients (not visible)	JPEG photos
Wavelet Coding	Wavelet transform -> Quantize -> code	Better quality at high compression than DCT	JPEG2000, fingerprints
Vector Quantization	Map image blocks to nearest codeword in dictionary	Approximate blocks with standard patterns	Video games, early video

Topic D3: JPEG & MPEG Standards

■ How JPEG Works — Step by Step

Every time you save a photo as .jpg on your phone, these exact steps happen inside:

Step	What happens	Why?
1. Color Conversion	RGB to YCbCr (brightness + 2 colour channels)	Human eye is more sensitive to brightness than colour
2. Chroma subsampling	Reduces colour channels to half resolution	Eye barely notices colour at fine detail
3. Blocking	Cuts image into 8x8 pixel squares	Standard block size for DCT
4. Apply 2D DCT	Turns each 8x8 block to 64 frequency values	Concentrate energy in low frequencies
5. Quantization	Divide each DCT value by quantization table, then round to nearest 0	This is the LOSSY step — high-freq values become 0
6. Zig-zag scan	Reads the 8x8 block in a zig-zag order into 1D list	Groups zeros together (mostly at the end)
7. RLE and DC values	Encodes values of zeros efficiently	Most AC values are 0 after quantization
8. Huffman coding	Encodes final values with variable-length codes	Final size reduction

Quality setting in JPEG controls Step 5 — higher quality = less quantization = bigger file but less loss.

■ MPEG — Extending JPEG to Video

Video = many images (frames) per second. MPEG extends JPEG by also removing **temporal redundancy** — consecutive frames in a video are usually very similar!

■ Example: A news anchor talking — the background stays the same for seconds. MPEG only encodes what **CHANGED** between frames, not the whole frame every time.

Frame Type	Full name	How it's coded	Analogy
I-frame	Intra frame	Coded completely by itself like a JPEG	Full photo
P-frame	Predicted frame	Only changes from previous I or P frame	List of edits from last photo
B-frame	Bidirectional frame	Changes predicted from both previous AND next frame	Changes between 2 reference photos

- **Motion Estimation:** Divide frame into 16x16 blocks. Find where each block moved in the reference frame. Store just the movement vector + leftover difference.
- **GOP (Group of Pictures):** Pattern like IBBPBBPBBP... I-frames allow random access (skip to any point in video).
- **MPEG-1:** CD-ROM video. **MPEG-2:** DVD, broadcast TV. **MPEG-4:** Internet streaming.
- **H.264/AVC:** Half the size of MPEG-2 — used in YouTube, Blu-ray. **H.265/HEVC:** Half of H.264 — used for 4K.

UNIT E — Image Segmentation and Image Representation

Topic E1: Edge Detection, Thresholding & Region-Based Segmentation

■ What is Segmentation? — The Big Picture

Segmentation = dividing an image into meaningful parts. Instead of random pixels, you want to identify 'this is the car', 'this is the road', 'this is the sky'.

■ *Real-life uses: Tesla Autopilot (detect road/pedestrians), medical imaging (find tumour in MRI scan), face unlock on your phone.*

Method 1 — Edge Detection (find boundaries between objects):

Detector	How it works	Key formula / kernel	Best for
Sobel	Gradient in X and Y direction	$G_x = [-1, 0, 1; -2, 0, 2; -1, 0, 1]$, $G_y = \text{transpose}(G_x)$	General edge detection
Prewitt	Same as Sobel, equal weights	$G_x = [-1, 0, 1; -1, 0, 1; -1, 0, 1]$	Simple edges
Laplacian	Second derivative — finds zero crossings	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Precise edge location
LoG	Gaussian smooth THEN Laplacian	Smooth first, then find zero crossings	Robust to noise
Canny	4-step optimal detector	Gaussian + Gradient + Non-max suppression + Hysteresis	Best overall, used everywhere

Canny Detector — 4 Steps (must know for exam):

Step 1: **Smooth with Gaussian** — removes noise before detecting edges.

Step 2: **Compute gradient** (magnitude and direction) using Sobel at every pixel.

Step 3: **Non-maximum suppression** — thin edges to 1-pixel width by keeping only local maxima in gradient direction.

Step 4: **Double thresholding + Hysteresis** — Strong edges (above high threshold) are kept. Weak edges (between two thresholds) are kept ONLY IF connected to a strong edge. Below low threshold = discard.

■ *Canny is like a careful detective: it finds all possible edges (Step 2), thins them neatly (Step 3), then decides which are real vs noise (Step 4).*

Method 2 — Thresholding (simplest segmentation):

Pick a threshold T . Pixels brighter than T = white (foreground). Darker = black (background). Done!

Type	How T is chosen	Formula	When to use
Global	One T for whole image	$g = 255$ if $f \geq T$, else $g = 0$	When lighting is uniform
Otsu's Method	Auto-pick T to maximise between-class variance	$\text{sigma}_B^2(T) = w_0 * w_1 * (\mu_0(T) - \mu_1(T))^2$, maximise over T	Best automatic method
Adaptive	T changes for each region/pixel	$T = \text{local mean} + \text{offset}$	Uneven lighting (scanned docs)
Multi-level	Multiple T values for 3+ classes	Multiple thresholds	Segment multiple objects

Otsu's Method formula in plain English:

$$\text{sigma}_B^2(T) = w_0(T) * w_1(T) * (\mu_0(T) - \mu_1(T))^2$$

w_0 = fraction of pixels below T , w_1 = fraction above T . μ_0 = average brightness below T , μ_1 = average above T . Try all T values from 0-255, pick the one with the HIGHEST sigma_B^2 .

Method 3 — Region-Based Segmentation:

Method	How it works	Analogy
Region Growing	Start from a seed pixel. Keep adding neighbours that are similar to the seed.	Flood fill / MS Paint

Region Split & Merge	Start with whole image, split non-uniform regions into 4, merge similar adjacent regions.	Quadtree
Watershed	Treat image as mountain landscape. Flood from valleys. Watersheds meet at boundary.	Water flooding
K-means Clustering	Group pixels into K clusters by colour/brightness similarity. Start with adjustable marbles into K buckets.	Statistical

Topic E2: Boundary Representation — Chain Codes, Polygonal Approx & Segments

■ Representing a Boundary as Data

Once you've found an object's boundary (edge), you need to **store/describe it as data**. Three ways to do this:

1. Chain Code (Freeman Chain Code):

Walk along the boundary pixel by pixel. At each step, record which direction you moved.

Type	Direction codes	Example sequence	Properties
4-directional	0=Right, 1=Up, 2=Left, 3=Down	0,0,1,1,2,2,3,3 (a rectangle)	Simpler, fewer codes
8-directional	0-7 (all 8 compass directions)	More precise boundary tracing	More common, diagonal steps allowed

- **Normalise chain code:** Rotate the starting sequence to find the minimum value version → removes start-point dependency.
- **Difference chain code:** Instead of absolute direction, store the CHANGE in direction between steps → rotation invariant.
- **Drawback:** Sensitive to noise — one noisy pixel changes the whole code.

2. Polygonal Approximation:

Instead of storing every boundary pixel, approximate the shape with straight line segments (a polygon).

■ *Like drawing a star — instead of the exact curve, you use straight lines between key points. Fewer line segments = more compressed but less accurate.*

- **Splitting method (Ramer-Douglas-Peucker):** Draw a straight line between two endpoints. Find the farthest boundary point from this line. If farther than threshold, split and recurse.
- **Merging method:** Start with each pixel as its own segment. Merge adjacent segments if the straight-line error is below a threshold.
- **Error metric:** Maximum perpendicular distance between actual boundary and the approximating line.

3. Boundary Segments (Signatures):

- **Boundary signature:** Convert 2D boundary into 1D signal — e.g. plot distance from centroid vs. angle $r(\theta)$.
- **Convex hull:** Smallest convex polygon that wraps around the shape (like wrapping a rubber band around it).
- **Convex deficiencies:** Gap areas between the boundary and its convex hull — used to identify concave features (like fingers in a hand outline).
- **Skeleton / Medial Axis:** Thin the shape down to a 1-pixel-wide 'skeleton' — like the bones of a shape.

Topic E3: Boundary Descriptors — Simple & Fourier Descriptors

■ Describing Shape with Numbers

Once you have a boundary, you want to describe the **shape** using numbers so you can compare, classify, or recognise objects.

Simple (Basic) Descriptors:

Descriptor	Formula	What it tells you	Invariant to?
------------	---------	-------------------	---------------

Perimeter	Count of boundary pixels	Total edge length	—
Area	Count of interior pixels	Size of object	—
Compactness	P^2 / A	How 'circle-like' the shape is. Circle = 4 * π (maximum)	Scale
Eccentricity	Major axis / minor axis of best-fit ellipse	How elongated/stretched the shape is	Rotation
Euler number	$E = \text{components} - \text{holes}$	Topology — does shape have holes?	Scale, rotation
Centroid	$x_bar = \text{mean}(x)$, $y_bar = \text{mean}(y)$	Centre of mass of the shape	—
Bounding box	Min/max x and y coordinates	Smallest rectangle containing shape	—

■ Fourier Descriptors — Shape as Waves

This is the coolest descriptor. You represent the boundary as a list of complex numbers ($x + jy$ for each boundary point), then take the **Fourier Transform of that sequence**. The result — the Fourier Descriptors — is a compact, powerful shape signature.

■ *Analogy: Like converting a song to MP3 — the Fourier Descriptors are the 'notes' that make up the shape's 'melody'. Use just the first few notes = rough shape. Add more notes = finer details.*

Fourier Descriptor Formula:

Boundary sequence: $s(k) = x(k) + jy(k)$ for $k=0,1,...,K-1$

Fourier Descriptors: $a(u) = \text{SUM}[k=0 \text{ to } K-1] s(k) * e^{(-j*2*\pi*u*k/K)}$

Reconstruct with P descriptors: use only $a(0), a(1), ..., a(P-1)$

Invariance needed	How to achieve it	What to do
Translation (position)	Ignore $a(0)$ — it only encodes the centroid position	Don't use $a(0)$
Scale (size)	Divide all by $ a(1) $	Normalise by $ a(1) $
Rotation	Use magnitudes $ a(u) $ only, ignore phase	Take abs value
Starting point	Magnitudes $ a(u) $ are unaffected	Take abs value

*Key insight: Use **more FDs** for precise shape matching. Use **fewer FDs** for a compact rough shape description. Compare shapes by comparing their FD vectors.*

EXAM QUICK REVISION — All Key Formulas in One Place

Topic	Formula / Rule	Remember it as...
Image storage	$M \times N \times k$ bits	Pixels x bits per pixel
Grey levels	2^k levels for k-bit image	8-bit = 256 levels
Nyquist rule	Sample $\geq 2 \times$ max frequency	At least twice as fast as detail
2D DFT	$F(u,v) = \sum \sum f(x,y) \cdot e^{-j2\pi(ux/M+vy/N)}$	Convert image to frequencies
FFT speedup	$O(N^2) \rightarrow O(N \log N)$	Massively faster DFT
DCT key idea	Pack energy into low-freq coeffs	Basis of JPEG
Gamma transform	$s = c \cdot r^\gamma$	$\gamma < 1$ = brighten, $\gamma > 1$ = darken
Histogram EQ	$s_k = (L-1) \cdot \text{CDF}(r_k)$	Stretch histogram to fill 0-255
Spatial convolution	$g(x,y) = \sum w(s,t) \cdot f(x+s,y+t)$	Slide kernel over image
Convolution theorem	$\text{FT}\{f \text{ convolved } h\} = F \cdot H$	Blurring in space = multiply in freq
Gaussian LPF	$H = e^{-(D^2 / 2 \cdot \sigma^2)}$	No ringing artefacts
Laplacian kernel	$[0,1,0; 1,-4,1; 0,1,0]$	2nd derivative, edge detection
Unsharp masking	$g = f + k \cdot (f - \text{blurred}_f)$	Add back the sharpness
Degradation model	$g = h \text{ convolved } f + \text{noise}$	Observed = blurred original + noise
Wiener filter	$F_{\text{hat}} = [H^* / (H ^2 + S_n/S_f)] \cdot G$	Smart inverse filter
Huffman key idea	Frequent values get shorter codes	Short code for common symbols
RLC key idea	Replace run of N identical values with (value, N)	AAAAAAA $\rightarrow$ (A,7)
JPEG pipeline	RGB $\rightarrow$ YCbCr $\rightarrow$ 8x8 blocks $\rightarrow$ DCT $\rightarrow$ Quantize $\rightarrow$ Zigzag $\rightarrow$ RLE $\rightarrow$ Huffman	Block + Huffman
MPEG frame types	I=full, P=from prev, B=from prev+next	I-frame is a keyframe
Otsu threshold	Maximise $\sigma_B^2 = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$	Max between-class variance
Canny 4 steps	Gaussian $\rightarrow$ Gradient $\rightarrow$ NonMaxSup $\rightarrow$ Hysteresis	Smooth $\rightarrow$ Detect $\rightarrow$ Thin $\rightarrow$ Filter
Chain code	Sequence of direction codes (0-7)	Walk the boundary, record direction
Compactness	P^2 / A	Circle has smallest value ($4 \cdot \pi$)
Fourier descriptors	$s(k) = x+jy$, take 1D DFT	Use $ a(u) $ for rotation invariance

You've got this! Every concept is understandable — just re-read the analogies if something feels unclear. Good luck! ■